Analysis of Speech Emotion Recognition and Detection using Deep Learning

Rajeev Ranjan
Department of Electronics and Communication Engineering
Chandigarh University, Mohali, Punjab, India
rajeevranjan1134@gmail.com
https://orcid.org/0000-0003-2359-4879

2022 IEEE Conference on Interdisciplinary Approaches in Technology and Management for Social Innovation (IATMSI) | 978-1-6654-7719-2/22/$31.00 ©2022 IEEE | DOI: 10.1109/IATMSI56455.2022.10119297

Abstract— **Speech emotion recognition (SER) is a mechanism to identify emotions from speech or voice. With the help of SER, robots can understand human feelings. In this challenging world, SER is one of the latest inventions that help to know about human emotion when saying words. Before the widespread use of deep learning, SER relied on various approaches, including support vector machines (SVM) and hidden Markov models (HMM) with several distinct and preprocessing technical features. SER is a challenging task of computational human interaction. This topic has gotten so much attention in the past couple of years. Numerous techniques have been used in speech emotion recognition to extract emotions from voice signals, including several well-developed speech examinations and classification methods. In the traditional way of speech emotion, recognition features are extracted from the speech signals. Then the features are selected, which is collectively known as the selection module, and then the emotions are recognized. This is a very lengthy and time taking process. This paper design an algorithm based on feature extraction and model creation that recognizes the emotion based on the deep learning technique.**

Keywords— Speech emotion recognition; deep learning; support vector machines; Hidden Markov models.

I. INTRODUCTION

SER is complicated because emotions are subjective, and audio is challenging to interpret. It is well recognized that people's emotional states can cause specific physical changes in their bodies. Moreover, It is obligatory to grasp its application and its part in emotion. Because of it, this paper aims to understand deep learning methods for SER, from databases to models. H. Cao et al. discussed emotion detection to tackle the issue of classification using SVM [1] and a three-level SER technique to improve speaker-independent SER [2]. The method of discrete Log Frequency Power Coefficients (DFPC) and short time (LFPC) is used for categorizing speech signals [3]. The modulation spectral features (MSFs) algorithm for recognizing human voice emotion was discussed in [4]. The described ensemble random forest to trees (ERFT) method is used for emotion identification [5]. Acoustic-prosodic (AP) characteristics were used in a fusion-based technique to recognize speech emotion. Domain-specific emotion identification identifies both negative and positive emotions were discussed in [7]. Yang et al. described the harmonic features for understanding speech emotions and psychoacoustic perception from music theory [8]. Spectral property to characterize groups and identify emotions and a computational system for emotion recognition was discussed [9,10]. Binary decision trees suggested a hierarchical structure for emotion recognition domains [11-13]. A computational technique for emotion identification and classification issue of speech emotion recognition the Gaussian mixture vector autoregressive (GMVAR) approach, a blend of GMM and autoregressive, has been developed [14-16].

II. METHODOLOGY

We have used the dataset TESS (Toronto Emotional Speech Set). In a single set there are 200 samples of one emotion with the words spoken were "Say the word 'by two persons and recordings were made the set of all seven emotions (anger, disgust, fear, happiness, pleasant surprise (SP), sadness, and neutral). The extension of the audio file is '.WAV'. There are 2700 audio files that were used to train the LSTM model. The model will go through all recorded samples. The dataset files are arranged in such a manner so that both actress files can be in different folders with their all emotions recording samples. Initially, we have imported some libraries:

```
import pandas as pd #data analysis
import numpy as np # mathematical operations on arrays
import os # manage directory
import seaborn as sns   #to generate statistical graph
import matplotlib. pyplot as plt # to make graphs
import librosa      #music and audio analysis
import librosa.display# to display waveshow and spectogram
from IPython.display import Audio
import warnings    # to hide warnings
warnings. filterwarnings('ignore')
```

The first five i.e. pandas, numpy, os, seaborn, and matplotlib visualization. We will use the librosa and librosa.display to import the audio files. If you want to play the audio, we have to import the ipython.display import audio to ignore the warnings, we are using warnings.filter warnings ('ignore'). Now loading the dataset:

```
paths = []; labels = []
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
paths.append(os.path.join(dirname, filename))
        label = filename.split('_')[-1]
```

978-1-6654-7719-2/22/$31.00 ©2022 IEEE

```
        label = label.split('.')[0]
labels.append(label.lower())
    if len(paths) == 2800:
        break
print('Dataset is Loaded')
```

So all the paths are displayed using this syntax for dirname,_ , filenames in os.walk ('/kaggle/input'): Using paths and labels we have created two lists and by these lists we will create a data frame by adding everything into the list. With the paths we are going to load the audio In order to get the label alone from the whole path of the different file we are using the label column. Label.append will move the labels to the label list.

```
len(paths)
paths[:5]; labels[:5]
## Create a dataframe
df = pd.DataFrame()
df['speech'] = paths
df['label'] = labels
df.head()
df['label'].value_counts()
```

EXPLORATORY DATA ANALYSIS

```
sns.countplot(df['label'])
```

Here we have created a data frame for paths and labels. The total number of samples use for each emotion i.e. fear, anger, disgust, neutral sad, ps, and happy are 400. Total 2800 samples.

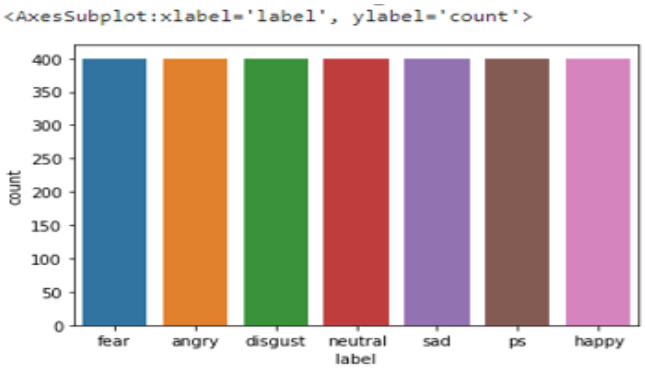


Fig. 1. Different emotions

Now we will display the waveform of the audio files for this:

```
def waveshow(data, sr, emotion):
plt.figure(figsize=(10,4))
plt.title(emotion, size=20)
librosa.display.waveshow(data, sr=sr)
plt.show()
def spectogram(data, sr, emotion):
    x = librosa.stft(data)
    xdb = librosa.amplitude_to_db(abs(x))
plt.figure(figsize=(11,4))
plt.title(emotion, size=20)
librosa.display.specshow(xdb, sr=sr, x_axis='time', y_axis='hz'); plt.colorbar()
```

We have created functions here named wave show and spectrogram by giving suitable Dimensions and lables now we will use these functions to display the waveform and spectrogram of different emotions for this we will have to create a snippet for different emotions:

For example, for fear:

```
emotion = 'fear'
path = np.array(df['speech'][df['label']==emotion])[0]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```

Filtered the path of fear emotion in this section

(path = np.array(df['speech'][df['label']==emotion])[0])

For the sampling rate

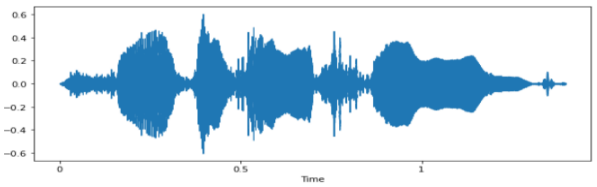
(data, sampling_rate = librosa.load(path))

To hear the audio here Audio(path) is used

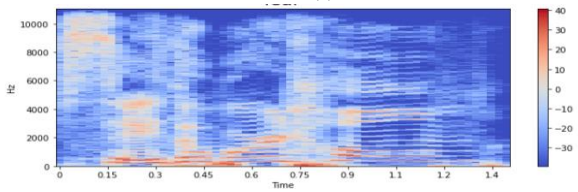
Then calling the functions:

The results obtained are as follows:

A. FEAR



(a)

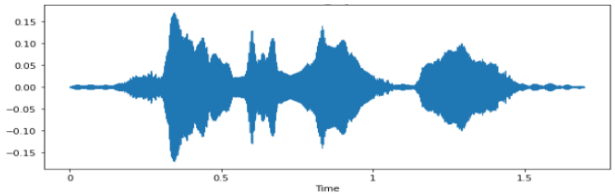


(b)

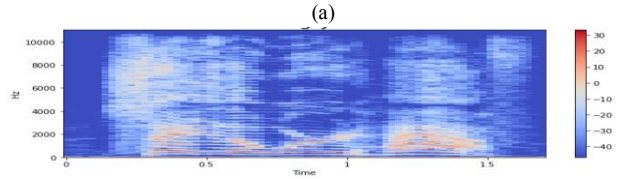
Fig. 2. a) Waveform for fear emotion b) Spectrogram for fear emotion.

B. ANGRY

```
emotion = 'angry'
path = np.array(df['speech'][df['label']==emotion])[1]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)



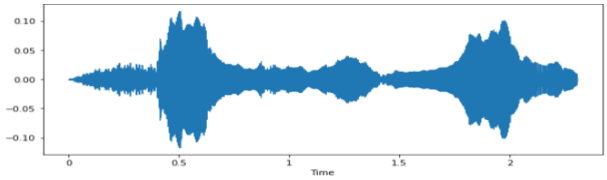
(b)

Fig. 3. a) Waveform for angry emotion b) Spectrogram for angry emotion

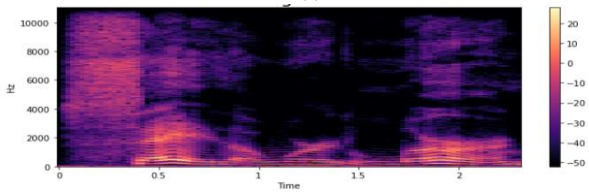
C. DISGUST

```
emotion = 'disgust'
```

```
path = np.array(df['speech'][df['label']==emotion])[1]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)

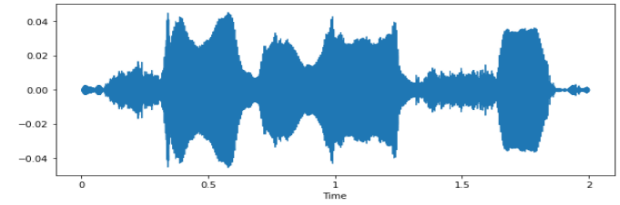


(b)

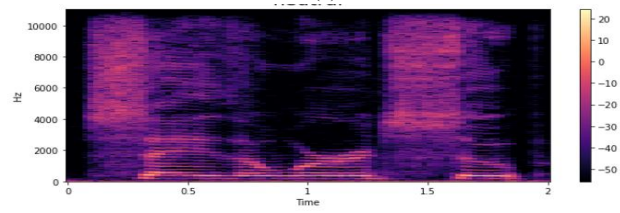
Fig. 4. a) Waveform for disgust emotion b) Spectrogram for disgust emotion

D. NEUTRAL

```
emotion = 'neutral'
path = np.array(df['speech'][df['label']==emotion])[0]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)

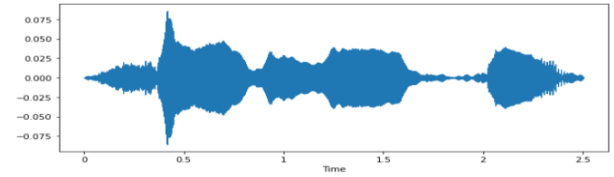


(b)

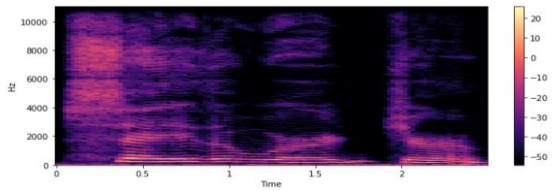
Fig. 5. a) Waveform for neutral emotion b) Spectrogram for neutral emotion

E. SAD

```
emotion = 'sad'
path = np.array(df['speech'][df['label']==emotion])[0]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)

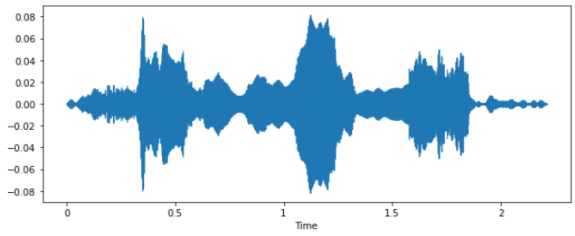


(b)

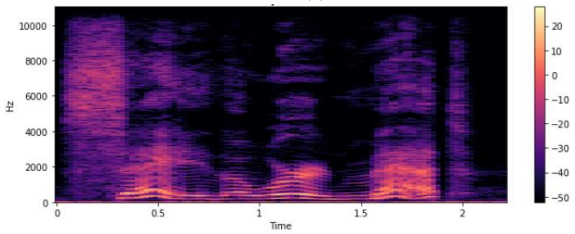
Fig. 6. a) Waveform for sad emotion b) Spectrogram for sad emotion

F. PS

```
emotion = 'ps'
path = np.array(df['speech'][df['label']==emotion])[0]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)

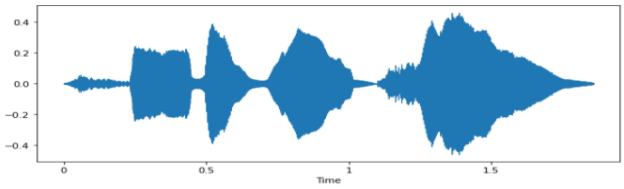


(b)

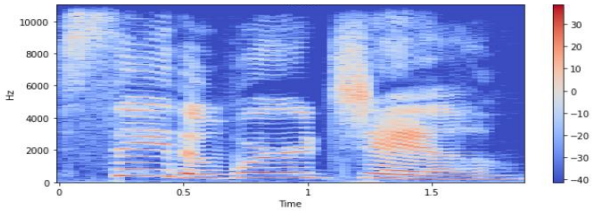
Fig. 7. a) Waveform for PS emotion b) Spectrogram for PS emotion

G. HAPPY

```
emotion = 'happy'
path = np.array(df['speech'][df['label']==emotion])[0]
data, sampling_rate = librosa.load(path)
waveshow(data, sampling_rate, emotion)
spectogram(data, sampling_rate, emotion)
Audio(path)
```



(a)



(b)

Fig. 8. a) Waveform for happy emotion b) Spectrogram for happy emotion

Feature extraction is the process of extracting various acoustic features of the speech which can directly affect the accuracy of the classification results, acoustic features include physical aspects of spoken language that can be recorded and analyzed these include waveform analysis, FFT or LPC analysis, voice onset time, format frequency, measurements and so on. For the feature extraction purpose, various methods are used such as MFCC, LPC, LPCC, LSF, PLP, and DWT. We are using the MFCC method in our model for the process of feature extraction. A Mel Frequency Cepstral Coefficient (MFCC) is basically one of the methods used to extract the various acoustic features of a speech from raw data to perform various things on the extracted data which can be further used for the classification of the emotions of a speech.

```
def extract_mfcc(filename):
    y, sr = librosa.load(filename, duration=3, offset=0.5)
    mfcc = np.mean(librosa.feature.mfcc(y=y, sr=sr, n_mfcc=40).T, axis=0)
    return mfcc
extract_mfcc(df['speech'][0])
```

So we have created here a function named extract MFCC and in which we have defined the duration of the feature extraction process as the various data have different lengths so we have set the duration to 3 secs and offset to 0.5 in the third step we have extracted the features of our data given in the filename.

III. Extracting features

```
X_mfcc = df['speech'].apply(lambda x: extract_mfcc(x))
```

Like in the previous step we have used the MFCC on only one sample here in this module we are going to apply the MFCC function on the whole sample that is our whole 2700 samples, and this is one of the major steps in this whole process. After this step we will get the data in some sequential form.

To convert the data into the 3D array form as used by the LSDM model we will use the following steps:

```
X = [x for x in X_mfcc]
X = np.array(X)
X.shape
## input split
X = np.expand_dims(X, -1)
X.shape
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder()
y = enc.fit_transform(df[['label']])
```

For the preprocessing we have created and imported a hot encoder here:

Creating the LSDM model:

```
y = y.toarray()
y.shape
from keras.models import Sequential
from keras.layers import Dense, LSTM, Dropout
model = Sequential([
LSTM(256,return_sequences=False, input_shape=(40,1)),
Dropout(0.2),
Dense(128, activation='relu'),
Dropout(0.2),
Dense(64, activation='relu'),
Dropout(0.2),
Dense(7, activation='softmax') ] )
model.compile(loss='categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
model.summary()
model.compile(loss='categorical_crossentropy',
optimizer='adam', metrics=['accuracy'])
model.summary()
```

For creating the LSTM model we have to import the sequential dense lstm and the dropout form keras.models and keras.layers respectively. Then specifying the various values required. In the next step compiling the model for loss and accuracy and then model.

Training the model:

```
# Train the model
history = model.fit(X, y, validation_split=0.2, epochs=50, batch_size=512)
```

Validation split = it will do the splitting for us.

Epochs = the number of complete passes through the complete dataset.

Batch size= the number of training examples utilized in one literation.

IV. RESULTS AND DISCUSSION

Using the matplotlib library we have visualized the accuracy and loss of the model.

```
epochs = list(range(50))
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
plt.plot(epochs, acc, label='train accuracy')
plt.plot(epochs, val_acc, label='val accuracy')
plt.xlabel('epochs'); plt.ylabel('accuracy')
plt.legend(); plt.show()
```

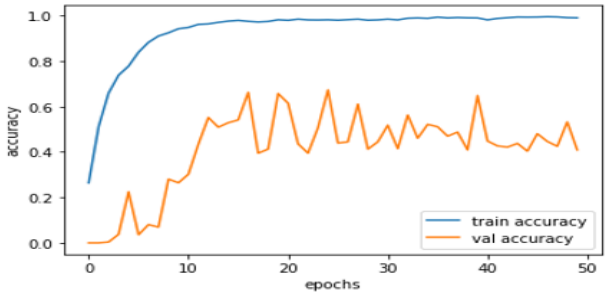


Fig. 9. Accuracy models

```
loss = history.history['loss']
val_loss = history.history['val_loss']
plt.plot(epochs, loss, label='train loss')
plt.plot(epochs, val_loss, label='val loss')
plt.xlabel('epochs'); plt.ylabel('loss')
plt.legend(); plt.show()
```

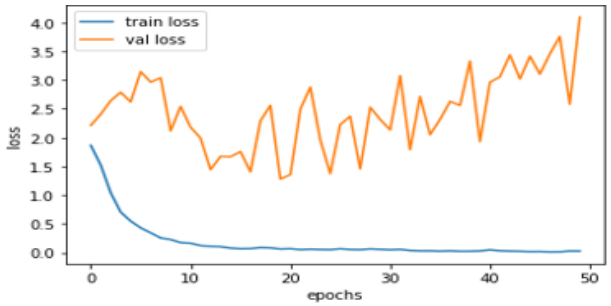


Fig. 10. Loss of the model

So, in this model after analyzing 2800 samples of data using the MFCC method of feature extraction and through the LSTM model for training and testing purposes we have got the accuracy of the model of about 70 percent and a value loss of around 30 percent. Further, we can increase the accuracy of the model by using more feature extraction techniques like delta MFCC and double delta MFCC and by giving more data to the model like in our case we have used 2800 samples of data for the so if the number of samples used is increased say 5600 the accuracy of the model will increase. In Table 1 is shown that the analysis for different data.

Table1: Running the model by giving different data and the results

TEST CASE	VAL ACCURACY (%)	VAL LOSS (%)
1	73	27
2	70	30
3	68	32
4	70	30
5	75	25
6	70	30
MEAN	71 %	29%

So, the average accuracy of this model is found to be 71%.

V. CONCLUSION

In this paper, we have tried to analyze some samples of speech using the deep learning technique. Firstly, we loaded the datasets then we visualized the different human emotions using our functions wave show and spectrogram using the Librosa library. Then we extracted the acoustic features of all our samples using the MFCC method and arranged the sequential data obtained in the 3D array form as accepted by the LSTM model. Then we build the LSTM model and after training the model we visualized the data into the graphical form using the matplotlib library and after some repeated testing using different values, the average accuracy of the model is found to be 71%.

REFERENCES

[1] H. Cao, R. Verma, and A. Nenkova, “Speaker-sensitive emotion recognition via ranking: Studies on acted and spontaneous speech,” Comput. Speech Lang., vol. 28, no. 1, pp. 186–202, Jan. 2015.

[2] L. Chen, X. Mao, Y. Xue, and L. L. Cheng, “Speech emotion recognition: Features and classification models,” Digit. Signal Process., vol. 22, no. 6, pp. 1154–1160, Dec. 2012

[3] T. L. Nwe, S. W. Foo, and L. C. De Silva, “Speech emotion recognition using hidden Markov models,” Speech Commun., vol. 41, no. 4, pp. 603–623, Nov. 2003.

[4] S. Wu, T. H. Falk, and W.-Y. Chan, “Automatic speech emotion recognition using modulation spectral features,” Speech Commun., vol. 53, no. 5, pp. 768–785, May 2011.

[5] J. Rong, G. Li, and Y.-P. P. Chen, “Acoustic feature selection for automatic emotion recognition from speech,” Inf. Process. Manag., vol. 45, no. 3, pp. 315–328, May 2009.

[6] C.-H. Wu and W.-B. Liang, “Emotion Recognition of Affective Speech Based on Multiple Classifiers Using Acoustic-Prosodic Information and Semantic Labels,” IEEE Trans. Affect. Comput., vol. 2, no. 1, pp. 10–21, Jan. 2011.

[7] S. S. Narayanan, “Toward detecting emotions in spoken dialogs,” IEEE Trans. Speech Audio Process., vol. 13, no. 2, pp. 293–303, Mar. 2005.

[8] B. Yang and M. Lugger, “Emotion recognition from speech signals using new harmony features,” Signal Processing, vol. 90, no. 5, pp. 1415–1423, May 2010

[9] E. M. Albornoz, D. H. Milone, and H. L. Rufiner, “Spoken emotion recognition using hierarchical classifiers,” Comput. Speech Lang., vol. 25, no. 3, pp. 556–570, Jul. 2011.

[10] C.-C. Lee, E. Mower, C. Busso, S. Lee, and S. Narayanan, “Emotion recognition using a hierarchical binary decision tree approach,” Speech Commun., vol. 53, no. 9–10, pp. 1162–1171, Nov. 2011.

[11] W. Dai, D. Han, Y. Dai, and D. Xu, “Emotion Recognition and Affective Computing on Vocal Social Media,” Inf. Manag., Feb. 2015.

[12] Ranjan, Rajeev. "Speaker Recognition and Performance Comparison based on Machine Learning." Turkish Journal of Computer and Mathematics Education (TURCOMAT) 12, no. 14 (2021): 2297-2306.

[13] Ranjan, R., Jindal, N. & Singh, A.K. The identities of n-dimensional s-transform and applications. Multimed Tools Appl 81, 16661–16677 (2022). https://doi.org/10.1007/s11042-022-12757-8.

[14] Ranjan, Rajeev, Neeru Jindal, and A. K. Singh. "Multiplicative Filter Design Using S-Transform." International Conference on Micro-Electronics and Telecomm. Engineering (ICMETE), pp. 260-263, 2018.

[15] Ranjan, Rajeev, Neeru Jindal, and A. K. Singh. "A sampling theorem with error estimation for S-transform." Integral Transforms and Special Functions 30, no. 6 (2019): 471-491.

[16] Rajeev Ranjan, Abhishek. “Image encryption using discrete orthogonal Stockwell transform with fractional Fourier transform” *Multimedia Tools and Applications*, 2022. https://doi.org/10.1007/s11042-022-14240-w.